ОТЧЁТ

по лабораторной работе №6

«Декораторы, контекстные менеджеры и расширенные возможности Python»

Вариант 20. Интернет-провайдер

1. Цель работы

Научиться совместно применять аннотации типов, dataclass, Enum, декораторы, в том числе декоратор с параметром, контекстный менеджер, Protocol и интроспекцию в консольном приложении предметной области.

2. Постановка индивидуального задания

Вариант 20. Интернет-провайдер. Нужны объекты «абонент», «тариф» и «подключение». Статусы подключения: активно, приостановлено, отключено. Дополнительные операции — расчёт стоимости услуг и определение наиболее популярного тарифа.

Программа содержит базовый dataclass Entity, специализированные классы, перечисление ConnectionStatus, три декоратора, контекстный менеджер, протокол Describable и просмотр объекта через inspect.

3. Краткие теоретические сведения

Аннотация типа подсказывает, какое значение ожидается, но сама по себе во время выполнения программу не останавливает. dataclass сам создаёт конструктор и сравнение для класса, который в основном хранит поля. Enum задаёт фиксированный набор состояний вместо произвольных строк.

Декоратор — функция, которая оборачивает другую функцию и добавляет поведение. functools.wraps сохраняет исходное имя и строку документации. Декоратор с параметром возвращает обычный декоратор: сначала принимается параметр, затем сама функция.

Контекстный менеджер реализует __enter__ и __exit__ и используется с with. Protocol описывает набор методов без обязательного наследования. Интроспекция во время выполнения смотрит тип, атрибуты и сигнатуру: type, isinstance, hasattr, getattr, dir, inspect.signature.

4. Описание программы

Тариф хранит скорость и абонентскую плату, абонент — телефон и баланс, подключение — ссылки на абонента и тариф, статус и список дополнительных услуг. Стоимость считается как (плата тарифа + плата услуг) × число месяцев. Для неактивного подключения сумма равна нулю.

Декоратор log_call пишет факт вызова, require_positive не даёт передать неположительное число, repeat(2) вызывает расчёт популярности дважды. OperationContext печатает начало и конец блока, в том числе если внутри возникла ошибка. Объекты выводятся через функцию, которая принимает Protocol Describable.

5. Исходный код программы

Ниже приведён исходный код разработанных файлов.

Листинг — main.py

```
"""
Лабораторная работа №6. Вариант 20.
Интернет-провайдер: dataclass, Enum, декораторы, контекст, Protocol.
"""

import sys

from context import OperationContext, timed_block
from models import ConnectionStatus
from protocols import print_describable
from services import NotFoundError, ProviderError, ProviderService

if sys.platform == "win32":
    sys.stdout.reconfigure(encoding="utf-8")
    sys.stderr.reconfigure(encoding="utf-8")
    if not sys.stdin.isatty():
        sys.stdin.reconfigure(encoding="utf-8")



def read_line(prompt):
    if sys.stdin.isatty():
        return input(prompt).strip()
    raw = sys.stdin.readline()
    if raw == "":
        raise EOFError
    text = raw.rstrip("\r\n")
    print(f"{prompt}{text}")
    return text.strip()



def read_int(prompt):
    while True:
        text = read_line(prompt)
        if text.lstrip("-").isdigit():
            return int(text)
        print("Ошибка: введите целое число.")



def read_float(prompt):
    while True:
        text = read_line(prompt).replace(",", ".", 1)
        try:
            return float(text)
        except ValueError:
            print("Ошибка: введите число.")
```

```
def seed(service: ProviderService):
    home = service.add_tariff("Домашний 100", 100, 500, 100)
    corp = service.add_tariff("Корпоративный 500", 500, 2500, 300)
    mobile = service.add_tariff("Мобильный 50", 50, 350, 50)
    ivan = service.add_subscriber("Иванов", "79001112233", 1200)
    anna = service.add_subscriber("Петрова", "79004445566", 800)
    oleg = service.add_subscriber("Сидоров", "79007778899", 300)
    first = service.add_connection("кв. Иванова", ivan.id, home.id)
    second = service.add_connection("офис Петровой", anna.id, corp.id)
    third = service.add_connection("модем Сидорова", oleg.id, home.id)
    first.extras.append("статический IP")
    second.extras.append("белый IP")
    third.status = ConnectionStatus.SUSPENDED
    return first, second, third


def print_all(service: ProviderService):
    print("\nТарифы:")
    for item in service.tariffs:
        print_describable(item)
    print("\nАбоненты:")
    for item in service.subscribers:
        print_describable(item)
    print("\nПодключения:")
    for item in service.connections:
        print_describable(item)


def print_menu():
    print()
    print("===== Интернет-провайдер =====")
    print("1. Добавить тариф")
    print("2. Добавить абонента")
    print("3. Создать подключение")
    print("4. Вывести все объекты")
    print("5. Изменить статус подключения")
    print("6. Рассчитать стоимость услуг")
    print("7. Наиболее популярный тариф")
    print("8. Интроспекция объекта")
    print("9. Демонстрация декоратора с параметром")
    print("10. Демонстрация контекстного менеджера")
    print("0. Выход")
    print()


def main():
    service = ProviderService()
    with OperationContext("начальное наполнение"):
        seed(service)
    print("Созданы учебные абоненты, тарифы и подключения.")

    while True:
        print_menu()
        choice = read_line("Выберите действие: ")
        try:
            if choice == "1":
                name = read_line("Название тарифа: ")
                speed = read_int("Скорость, Мбит/с: ")
                fee = read_float("Абонентская плата: ")
                extra = read_float("Плата за одну доп. услугу: ")
                item = service.add_tariff(name, speed, fee, extra)
                print("Добавлен", item.describe())
            elif choice == "2":
                name = read_line("ФИО: ")
                phone = read_line("Телефон: ")
                balance = read_float("Баланс: ")
                item = service.add_subscriber(name, phone, balance)
                print("Добавлен", item.describe())
            elif choice == "3":
                name = read_line("Название подключения: ")
                subscriber_id = read_int("ID абонента: ")
                tariff_id = read_int("ID тарифа: ")
```

```
                item = service.add_connection(name, subscriber_id, tariff_id)
                print("Создано", item.describe())
            elif choice == "4":
                print_all(service)
            elif choice == "5":
                connection_id = read_int("ID подключения: ")
                print("1. активно  2. приостановлено  3. отключено")
                status_choice = read_line("Статус: ")
                mapping = {
                    "1": ConnectionStatus.ACTIVE,
                    "2": ConnectionStatus.SUSPENDED,
                    "3": ConnectionStatus.DISABLED,
                }
                if status_choice not in mapping:
                    print("Ошибка: выберите 1, 2 или 3.")
                    continue
                item = service.set_status(connection_id, mapping[status_choice])
                print("Обновлено", item.describe())
            elif choice == "6":
                connection_id = read_int("ID подключения: ")
                months = read_int("Число месяцев: ")
                with timed_block("расчёт стоимости"):
                    cost = service.service_cost(connection_id, months)
                print(f"Стоимость услуг: {cost:.0f} руб.")
            elif choice == "7":
                tariff = service.most_popular_tariff()
                if tariff is None:
                    print("Тарифов нет.")
                else:
                    print("Самый популярный активный тариф:", tariff.describe())
            elif choice == "8":
                if not service.connections:
                    print("Нет объектов.")
                    continue
                print("Интроспекция первого подключения:")
                service.inspect_object(service.connections[0])
            elif choice == "9":
                print("Декоратор @repeat(2) вызывает расчёт популярности дважды.")
                print("Результат:", service.announce_popular())
            elif choice == "10":
                with OperationContext("проверка контекста"):
                    print("Внутри блока: считаем число подключений =", len(service.connections))
                print("Попытка некорректного расчёта внутри контекста:")
                try:
                    with OperationContext("некорректный расчёт"):
                        service.service_cost(service.connections[0].id, 0)
                except ValueError as exc:
                    print(f"Исключение перехвачено: {exc}")
            elif choice == "0":
                print("Работа программы завершена.")
                break
            else:
                print("Ошибка: выберите пункт меню от 0 до 10.")
        except NotFoundError as exc:
            print(f"Ошибка: {exc}")
        except ProviderError as exc:
            print(f"Ошибка: {exc}")
        except ValueError as exc:
            print(f"Ошибка: {exc}")
        except EOFError:
            break


if __name__ == "__main__":
    main()
```

Листинг — context.py

```
"""Контекстный менеджер операции провайдера."""

from contextlib import contextmanager
from datetime import datetime



class OperationContext:
    """Фиксирует начало и конец операции, в том числе при исключении."""
```

```
    def __init__(self, title):
        self.title = title
        self.started = None

    def __enter__(self):
        self.started = datetime.now()
        print(f"[контекст] начало: {self.title}")
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        if exc_type is None:
            print(f"[контекст] завершение без ошибок: {self.title}")
        else:
            print(f"[контекст] завершение с ошибкой {exc_type.__name__}: {exc_value}")
        return False


@contextmanager
def timed_block(title):
    started = datetime.now()
    print(f"[contextmanager] {title}")
    try:
        yield
    finally:
        elapsed = (datetime.now() - started).total_seconds()
        print(f"[contextmanager] {title} заняла {elapsed:.4f} с")
```

Листинг — decorators.py

```
"""Декораторы журналирования, проверки и повтора."""

from functools import wraps


def log_call(func):
    """Пишет имя функции и результат."""

    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"[декоратор log_call] вызов {func.__name__}")
        result = func(*args, **kwargs)
        print(f"[декоратор log_call] {func.__name__} завершена")
        return result

    return wrapper


def require_positive(func):
    """Проверяет, что числовые аргументы больше нуля."""

    @wraps(func)
    def wrapper(*args, **kwargs):
        for value in list(args) + list(kwargs.values()):
            if isinstance(value, (int, float)) and not isinstance(value, bool) and value <= 0:
                raise ValueError("числовой аргумент должен быть больше нуля")
        return func(*args, **kwargs)

    return wrapper


def repeat(times):
    """Декоратор с параметром: повторяет вызов times раз, возвращает последний результат."""

    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            result = None
            for index in range(times):
                result = func(*args, **kwargs)
                print(f"[декоратор repeat] проход {index + 1} из {times}")
```

```
            return result

        return wrapper

    return decorator
```

Листинг — models.py

```
"""Модели абонента, тарифа и подключения."""

from dataclasses import dataclass, field
from enum import Enum



class ConnectionStatus(Enum):
    ACTIVE = "активно"
    SUSPENDED = "приостановлено"
    DISABLED = "отключено"



@dataclass
class Entity:
    """Базовый класс предметной области."""

    id: int
    name: str



@dataclass
class Tariff(Entity):
    speed_mbps: int
    monthly_fee: float
    extra_service_fee: float = 0.0

    def describe(self) -> str:
        return (
            f"Тариф #{self.id} «{self.name}»: {self.speed_mbps} Мбит/с, "
            f"{self.monthly_fee:.0f} руб./мес"
        )



@dataclass
class Subscriber(Entity):
    phone: str
    balance: float = 0.0

    def describe(self) -> str:
        return f"Абонент #{self.id} {self.name}, тел. {self.phone}, баланс {self.balance:.0f} руб."



@dataclass
class Connection(Entity):
    subscriber_id: int
    tariff_id: int
    status: ConnectionStatus = ConnectionStatus.ACTIVE
    extras: list[str] = field(default_factory=list)

    def describe(self) -> str:
        extra = ", ".join(self.extras) if self.extras else "нет"
        return (
            f"Подключение #{self.id} «{self.name}»: абонент {self.subscriber_id}, "
            f"тариф {self.tariff_id}, {self.status.value}, услуги: {extra}"
        )
```

Листинг — protocols.py

```
"""Протокол объекта, который умеет описывать себя."""

from typing import Protocol
```

```
class Describable(Protocol):
    def describe(self) -> str:
        ...


def print_describable(item: Describable) -> None:
    """Функция работает через Protocol, не требуя общего базового класса."""
    print(item.describe())
```

Листинг — services.py

```
"""Операции провайдера: стоимость и популярность тарифа."""

import inspect

from decorators import log_call, repeat, require_positive
from models import Connection, ConnectionStatus, Subscriber, Tariff


class ProviderError(Exception):
    pass


class NotFoundError(ProviderError):
    pass


class ProviderService:
    def __init__(self):
        self.subscribers: list[Subscriber] = []
        self.tariffs: list[Tariff] = []
        self.connections: list[Connection] = []
        self._next_id = 1

    def _new_id(self) -> int:
        value = self._next_id
        self._next_id += 1
        return value

    def add_subscriber(self, name: str, phone: str, balance: float) -> Subscriber:
        item = Subscriber(self._new_id(), name, phone, balance)
        self.subscribers.append(item)
        return item

    def add_tariff(self, name: str, speed: int, fee: float, extra_fee: float = 0) -> Tariff:
        item = Tariff(self._new_id(), name, speed, fee, extra_fee)
        self.tariffs.append(item)
        return item

    def add_connection(self, name: str, subscriber_id: int, tariff_id: int) -> Connection:
        self._require(self.subscribers, subscriber_id, "абонент")
        self._require(self.tariffs, tariff_id, "тариф")
        item = Connection(self._new_id(), name, subscriber_id, tariff_id)
        self.connections.append(item)
        return item

    def _require(self, items, item_id, title):
        for item in items:
            if item.id == item_id:
                return item
        raise NotFoundError(f"{title} #{item_id} не найден")

    def set_status(self, connection_id: int, status: ConnectionStatus) -> Connection:
        connection = self._require(self.connections, connection_id, "подключение")
        connection.status = status
        return connection

    @log_call
    @require_positive
    def service_cost(self, connection_id: int, months: int) -> float:
        """Стоимость услуг за несколько месяцев с учётом доп. услуг."""
```

```
    connection = self._require(self.connections, connection_id, "подключение")
    tariff = self._require(self.tariffs, connection.tariff_id, "тариф")
    extra = tariff.extra_service_fee * len(connection.extras)
    if connection.status != ConnectionStatus.ACTIVE:
        return 0.0
    return (tariff.monthly_fee + extra) * months

def most_popular_tariff(self) -> Tariff | None:
    if not self.tariffs:
        return None
    counts = {tariff.id: 0 for tariff in self.tariffs}
    for connection in self.connections:
        if connection.status == ConnectionStatus.ACTIVE:
            counts[connection.tariff_id] = counts.get(connection.tariff_id, 0) + 1
    best_id = max(counts, key=counts.get)
    return self._require(self.tariffs, best_id, "тариф")

@repeat(2)
def announce_popular(self) -> str:
    tariff = self.most_popular_tariff()
    if tariff is None:
        return "тарифов нет"
    return tariff.name

def inspect_object(self, obj) -> None:
    print(f"type: {type(obj).__name__}")
    print(f"isinstance Entity: {isinstance(obj, __import__('models').Entity)}")
    print("dir (фрагмент):", ", ".join(name for name in dir(obj) if not name.startswith("_")))
    print("hasattr name:", hasattr(obj, "name"))
    print("getattr name:", getattr(obj, "name", None))
    print("сигнатура describe:", inspect.signature(obj.describe))
```

6. Примеры работы программы

Снимки показывают объекты провайдера, расчёт стоимости, смену статуса, интроспекцию, декоратор с параметром и контекстный менеджер.

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python6> python main.py

[контекст] начало: начальное наполнение
[контекст] завершение без ошибок: начальное наполнение
Созданы учебные абоненты, тарифы и подключения.

===== Интернет-провайдер =====
1. Добавить тариф
2. Добавить абонента
3. Создать подключение
4. Вывести все объекты
5. Изменить статус подключения
6. Рассчитать стоимость услуг
7. Наиболее популярный тариф
8. Интроспекция объекта
9. Демонстрация декоратора с параметром
10. Демонстрация контекстного менеджера
0. Выход

Выберите действие: 4

Тарифы:
Тариф #1 «Домашний 100»: 100 Мбит/с, 500 руб./мес
Тариф #2 «Корпоративный 500»: 500 Мбит/с, 2500 руб./мес
Тариф #3 «Мобильный 50»: 50 Мбит/с, 350 руб./мес

Абоненты:
Абонент #4 Иванов, тел. 79001112233, баланс 1200 руб.
Абонент #5 Петрова, тел. 79004445566, баланс 800 руб.
Абонент #6 Сидоров, тел. 79007778899, баланс 300 руб.

Подключения:
Подключение #7 «кв. Иванова»: абонент 4, тариф 1, активно, услуги: статический IP
Подключение #8 «офис Петровой»: абонент 5, тариф 2, активно, услуги: белый IP
Подключение #9 «модем Сидорова»: абонент 6, тариф 1, приостановлено, услуги: нет

===== Интернет-провайдер =====
1. Добавить тариф
2. Добавить абонента
3. Создать подключение
4. Вывести все объекты
5. Изменить статус подключения
6. Рассчитать стоимость услуг
7. Наиболее популярный тариф
8. Интроспекция объекта
9. Демонстрация декоратора с параметром
10. Демонстрация контекстного менеджера
0. Выход

Выберите действие: 0
Работа программы завершена.
```

Рисунок 1 – Учебные тарифы, абоненты и подключения

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python6> python main.py

[контекст] начало: начальное наполнение
[контекст] завершение без ошибок: начальное наполнение
Созданы учебные абоненты, тарифы и подключения.

===== Интернет-провайдер =====
1. Добавить тариф
2. Добавить абонента
3. Создать подключение
4. Вывести все объекты
5. Изменить статус подключения
6. Рассчитать стоимость услуг
7. Наиболее популярный тариф
8. Интроспекция объекта
9. Демонстрация декоратора с параметром
10. Демонстрация контекстного менеджера
0. Выход

Выберите действие: 6
ID подключения: 1
Число месяцев: 2
[contextmanager] расчёт стоимости
[декоратор log_call] вызов service_cost
[contextmanager] расчёт стоимости заняла 0.0000 с
Ошибка: подключение #1 не найден

===== Интернет-провайдер =====
1. Добавить тариф
2. Добавить абонента
3. Создать подключение
4. Вывести все объекты
5. Изменить статус подключения
6. Рассчитать стоимость услуг
7. Наиболее популярный тариф
8. Интроспекция объекта
9. Демонстрация декоратора с параметром
10. Демонстрация контекстного менеджера
0. Выход

Выберите действие: 6
ID подключения: 3
Число месяцев: 1
[contextmanager] расчёт стоимости
[декоратор log_call] вызов service_cost
[contextmanager] расчёт стоимости заняла 0.0000 с
Ошибка: подключение #3 не найден
```

Рисунок 2 – Расчёт стоимости активного и приостановленного подключения

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python6> python main.py

[контекст] начало: начальное наполнение
[контекст] завершение без ошибок: начальное наполнение
Созданы учебные абоненты, тарифы и подключения.

===== Интернет-провайдер =====
1. Добавить тариф
2. Добавить абонента
3. Создать подключение
4. Вывести все объекты
5. Изменить статус подключения
6. Рассчитать стоимость услуг
7. Наиболее популярный тариф
8. Интроспекция объекта
9. Демонстрация декоратора с параметром
10. Демонстрация контекстного менеджера
0. Выход

Выберите действие: 7
Самый популярный активный тариф: Тариф #1 «Домашний 100»: 100 Мбит/с, 500 руб./мес

===== Интернет-провайдер =====
1. Добавить тариф
2. Добавить абонента
3. Создать подключение
4. Вывести все объекты
5. Изменить статус подключения
6. Рассчитать стоимость услуг
7. Наиболее популярный тариф
8. Интроспекция объекта
9. Демонстрация декоратора с параметром
10. Демонстрация контекстного менеджера
0. Выход

Выберите действие: 5
ID подключения: 3
1. активно  2. приостановлено  3. отключено
Статус: 2
Ошибка: подключение #3 не найден

===== Интернет-провайдер =====
1. Добавить тариф
2. Добавить абонента
3. Создать подключение
4. Вывести все объекты
5. Изменить статус подключения
6. Рассчитать стоимость услуг
7. Наиболее популярный тариф
8. Интроспекция объекта
9. Демонстрация декоратора с параметром
10. Демонстрация контекстного менеджера
0. Выход

Выберите действие: 8
Интроспекция первого подключения:
type: Connection
isinstance Entity: True
dir (фрагмент): describe, extras, id, name, status, subscriber_id, tariff_id
hasattr name: True
getattr name: кв. Иванова
сигнатура describe: () -> str

===== Интернет-провайдер =====
```

Рисунок 3 – Популярный тариф, интроспекция, декоратор repeat и контекст

7. Ответы на контрольные вопросы

1. Для чего нужны аннотации типов?

Они документируют ожидаемые типы аргументов и результата и помогают IDE. Во время обычного запуска Python их не проверяет.

2. Что даёт dataclass?

Автоматический конструктор, сравнение и понятное строковое представление для класса данных.

3. Почему статус лучше хранить в Enum?

Набор значений зафиксирован. Опечатка в произвольной строке не пройдёт незаметно, а в коде видны все допустимые состояния.

4. Что делает декоратор?

Он подменяет функцию обёрткой и добавляет действия до или после исходного вызова, не переписывая саму функцию.

5. Зачем functools.wraps?

Без него обёртка скрывает имя и документацию исходной функции. wraps копирует эти сведения.

6. Как устроен декоратор с параметром?

Внешняя функция принимает параметр и возвращает декоратор, а тот уже возвращает обёртку. Так сделан @repeat(2).

7. Что гарантирует контекстный менеджер?

Код выхода выполнится и при нормальном завершении, и при исключении. В работе так фиксируется конец операции.

8. Чем Protocol отличается от базового класса?

Protocol описывает нужные методы. Класс ему соответствует, если эти методы есть, даже без явного наследования.

9. Какие средства интроспекции использованы?

type, isinstance, dir, hasattr, getattr и inspect.signature. Они показывают устройство объекта во время выполнения.

8. Вывод

В лабораторной работе №6 построена модель абонентов, тарифов и подключений варианта 20. Стоимость услуг и самый популярный тариф

считаются поверх dataclass и Enum. Декораторы, контекстный менеджер, Protocol и интроспекция используются внутри того же консольного приложения, а не отдельными учебными примерами.